Pulmonary Embolism Detection

Aditya Mohan, Omid Chehreh Gosha

***Abstract*—Pulmonary embolisms (PE) are life-threatening diseases that can be diagnosed using Computed Tomography Pulmonary Angiography (CTPA). While rapid diagnosis and immediate treatment are highly important for alleviating the damage that PEs can cause, PEs remain among diagnoses that are frequently delayed. In this work, we present a Deep-Learning model, based on Convolutional Neural Networks (CNNs), that is capable of detecting and classifying Pulmonary Embolisms using CTPA scans. The work uses the dataset provided by the Radiological Society of North America (RSNA), which consists of scans collected from multiple hospitals across North America and made available as part of the Kaggle Competition on the same topic. Our model uses Xception, a state-of-the-art CNN, for feature extraction and then uses these features to classify all the metrics associated with the studies through transfer learning. This model can perform classification on most metrics with an accuracy above 90% and only diminishes in performance to 80% on heart-related metrics.**

***Keywords*—Pulmonary Embolisms, Convolutional Neural Networks, Image Recognition, Transfer Learning**

I. Introduction

A pulmonary embolism (PE) is a sudden blockage in a lung artery. It usually happens when a blood clot breaks loose and travels through the bloodstream to the lungs, and it can be life-threatening, especially if a clot is large or if there are many clots. To make a diagnosis, the healthcare provider needs the medical history of the patient, do a physical examination, and perform some blood and imaging tests. This is usually a very time-consuming process and is prone to over-diagnosis. Currently, CT Pulmonary Angiography (CTPA)[4], is the most common type of medical imaging techniques to evaluate patients with suspected PE. These CT scans consist of hundreds of images that require detailed review to identify clots within the pulmonary arteries. Thus, there exists a niche where Machine Learning could be applied to this analysis to detect PE symptoms based on clot detection on the CT scans. This is what we aim to do in this project.

The purpose of this work is to develop and evaluate an end-to-end deep learning model capable of detecting PE using the CTPA dataset provided by the RSNA. The dataset contains over 7200 CTPA images and corresponding labels for training that have been recorded from real patients across North America. The input to our pipeline is a pre-processed version of these images that has the relevant pixels in defined ranges through a process called windowing, explained further in section IV. The pipeline extracts features from this image using state-of-the-art convolutional neural networks, which are then further used for categorization over nine labels that correspond to each image on the image and study level. The output is a binary classification over each label, which is then used to compare with the targets and calculate respective losses and accuracies.

This report is structured as follows: In section II, a literature review of work in this domain is presented. Section III explores the dataset used in this work and the pre-processing that has been performed on this dataset. Section IV describes the pipeline in detail, while section V describes the results. Finally, section VI presents the conclusion and future outlook.

II. Related work

Machine learning (ML) is one of the best ways to integrate, analyze, and make predictions based on large, heterogeneous data sets [15]. Medical diagnosis via image processing and machine learning is considered one of the most important issues of Artificial Intelligence systems [1].

In applying ML to the problem of PE detection, the earliest work focused on using clinical variables or perfusion imaging rather than CTPA as inputs to Neural Networks, which led to moderate generalization [18]. Other early methods approach automated detection through complex feature engineering and pre-processing without external validation, and thus, were costly in more than one aspect [13]. Application of Deep-Learning removes or reduces the need for manual feature engineering on images and allows more flexibility in designing the end-end pipeline for this automated process by engineering the learning architectures directly.

More recent approaches have started focusing on using CTPA imaging for PE diagnosis. This is beneficial since CTPA imaging is the most commonly used imaging method for PE examinations, and the collection of data enables definitive diagnosis using different kinds of learning approaches. Moreover, the emerging area of applying Deep Learning techniques to medical applications enables setting-up a gradually improving process of collaboration between doctors and machines by delegating the rudimentary diagnosis to be performed by automated DL pipelines and further examination done by doctors. Thus, even if the model does not have the best accuracy, it would certainly improve the diagnosis time by allowing doctors to understand better the directions in which they should probe further. The work done in [16] relies on heavily pre-processing features of images via segmentation and vessel-alignment, which is then fed as an input to a 3D CNN, while the work done in [21] sub-divides each CTPA into small cubes, instead of working with a full CTPA scan. Our approach requires significantly less feature engineering, only limited to extracting the important features from the images through windowing and applying rudimentary transformations on it, and it works on full CT scans instead of sub-dividing them. Thus, if our results are improved upon, they present a

better case of integrability with existing medical diagnostic pipelines.

III. Dataset and Preprocessing

For this project, we used the dataset provided by the Radiological Society of North America (RSNA) [14] that contains three parts:

- A training set that consists of 7279 studies, which corresponds to a total of 1,790,594 training images with Labels
- A test dataset consisting of 650 studies, which correspond to 146,853 images that have no labels.
- A private dataset consisting of 1517 studies that are used by Kaggle to evaluate submissions.

A study has multiple images grouped into series. Thus, each image is characterized by identifiers:

- **StudyInstanceUID:** Indicates a unique identifier for the study to which the image belongs
- **SeriesInstanceUID:** Indicates the series in the study to which the image belongs
- **SOPInstanceUID:** A unique identifier for the image itself, defining the slice of a patient's data pertaining to the chest (in the context of CT scans, there are several images taken at different locations and angles from the patient body called slices).

For each training image there are 14 labels, that belong to 3 levels:

- **Image-Level Labels** Label for each image. Here, the only label at the image-level is the one used to indicate whether a PE is present on the image or not.
- **Exam-Level Labels:** These labels are used to detect the labels corresponding to characteristics of the PE and include the labels corresponding to the ratio of the right valve to left valve (greater than or less than 1), location of the PE (left, right, center), nature of PE (acute, chronic or both) and a label to indicate whether the study was conclusive or not.
- **Informational Labels:** These labels are used to indicate whether the radiologists noted an issue with motion or contrast in the study, whether there were artifacts present or whether the thing diagnosed was something else other than PE.

We used the insights obtained in [10] to determine the labels that need to be predicted, and finally narrowed the labels down to 9: `acute_and_chronic_pe`, `central_pe`, `chronic_pe`, `indeterminate`, `leftsided_pe`, `pe_present_on_image`, `rightsided_pe`, `rv_lv_ratio_gte_1`, `rv_lv_ratio_lt`.

The images are stored in a Digital Imaging and Communications in Medicine (DICOM) format [2], which is an international standard related to the exchange, storage, and communication of digital medical images. A series in the study has 2D image slices of different patients. As explained in [19], these images are generated by administering Iodine to the patient and giving contrast to the chest. Thus, the raw pixels contain information from different instances of the phases of the exam and through different tissues. The density of tissues is measured in Hounsfield Units (HU) [9], where the scale is taken from the range of -1000 HU (for air) and 0 HU for water. CT scan images may have a different representation on disk versus in the memory. This issue is usually because DICOM images may have negative values, but they are commonly stored with an unsigned integer. Using the intercept and slope value from the DICOM data structure (which are provided by the hardware manufacturer), we can transform the values to HU units. Thus to extract the relevant information from the images, we manipulated the CT numbers through a process called windowing, where we limited the pixels to a range of values, thus, allowing better contrast for the lungs. As detailed in [5] and [20], windowing is typically done by defining a center point and a length of the window for filtering the pixels. In our exploratory data analysis using [7] as a reference, we noticed that the majority of HU values were in values around -1000 and 0, indicating that most of the area of an image is filled with air and water. In requirements with the architecture explained in IV, we performed windowing three times on each slice with three different pair of (center, width): (40, 400), (100, 700), and (-600, 1500) to isolate the lung tissue and generate a good contrast for the PE to be visible. These were stacked together to create a 3-channel Image, as shown in 1 that could be used by our feature extractor.

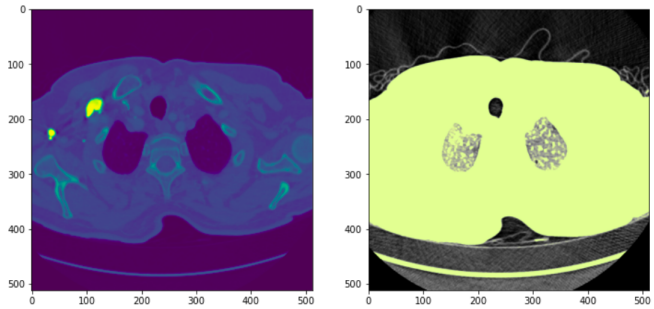

Fig. 1: comparison between the original DICOM Image (Left) and the 3-channel image generated after windowing (Right)

After windowing, we performed further augmentations on the images to improve the performance of our pipeline. We applied random rotations of 8 degrees in both directions and scaled the contrast of the image to better detect the PE. We applied random horizontal flip (with a 50% chance to flip the slice image) to images to better allow for better detection of the location of the PE and prediction on the ventricular ratios. The improvements are further discussed in Section V.

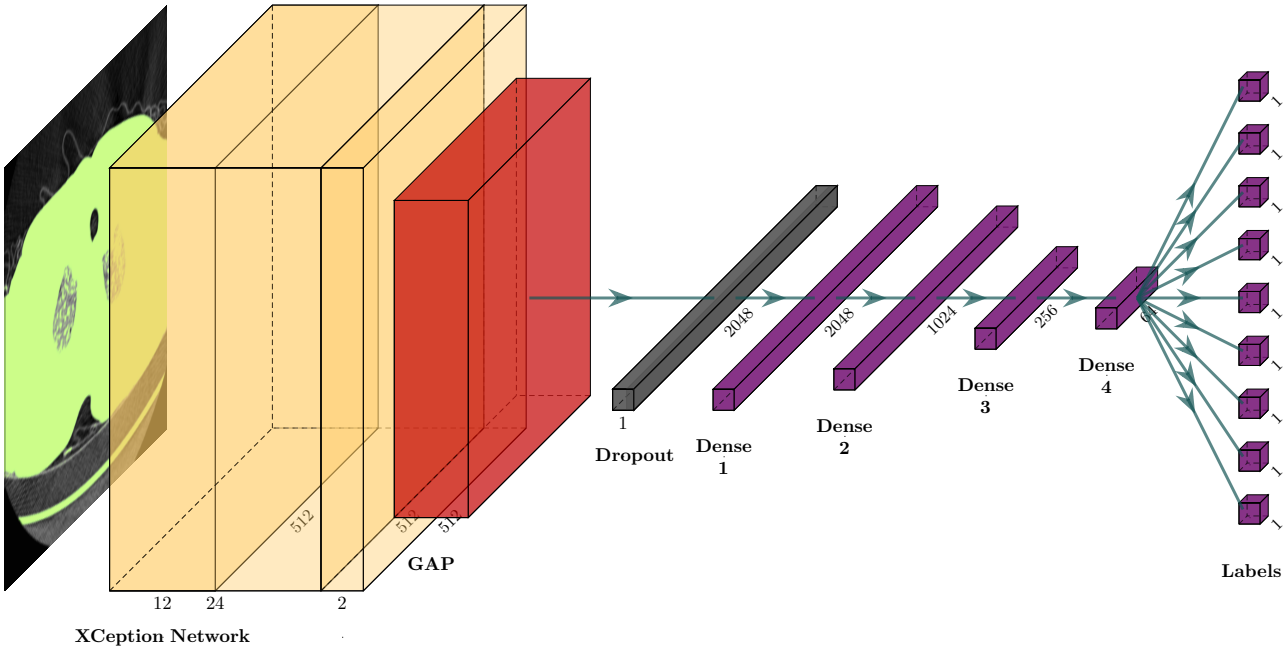

Fig. 2: Architecture employed for classification: The image is fed to the Xception network to extract latent features that are averaged using a Global Average Pooling Layer. After this, a dropout of 25% is applied, followed by 4 back to back fully connected layers that bring the features down to 64, which is then fed to 9 fully connected layers - 1 for each label - activated by sigmoid to predict the label probability

IV. Method

The Architecture is composed of two parts: (1) A convolutional base to extract the latent features from the image and (2) The classifier to predict the labels for this image. For the convolutional base, we used the Xception network [3] - a state of the art network developed by google - and removed its top fully connected layer. The feature extraction base of Xception consists of 36 convolutional layers, which are divided into 14 modules, all except the first and the last having linear residual components. The data is passed through three flows: The entry flow consists of 4 convolutional layers with batch-normalization and residual connection, the middle flow consists of 3 separable convolutions that are repeated 8 times, and the exit flow consists of 2 layers of normalized convolutions. The output of the Xception feature extractor is a 16x16x2048 vector of latent features. This is passed through a Global Average Pooling (GAP) Layer. This layer performs dimensionality reduction by taking the average of the 16x16 values and thus, producing a 2048 length feature vector. This feature vector is then passed through a dropout layer that randomly sets 25% of the 2048 values to 0. As explained in [8], the advantage this dropout layer offers is that the classifier is forced to learn more robust weights that are not affected by losing random Neurons, which helps stabilize the network and reducing over-fitting.

After the dropout, the feature vector is passed through four dense, fully connected layers that extract more specific features. The first dense layer converts the feature vector to half of its size, i.e., from 2048 to 1024, by learning the appropriate parameters. The second and the third dense layers decrease the feature vectors to 1/4 of their size, i.e., 1024 to 256, and then to 64. This 64 length vector is fed to a multi-output setup with nine outputs, each corresponding to 1 label. Each of these output layers is activated by a sigmoid function, in contrast to all the previous layers which have ReLu activation, and they combine the 64 features to output the probability of 1 label each.

The Xception network was pre-trained on the Imagenet Dataset [6] and is available in Keras as a module. We made its layers non-trainable during training. We used a test-train validation split of 0.8 for the other layers and a learning rate of 0.001. The model used the Adam optimizer [12], which calculates an exponential moving average of the gradient and the squared gradient and then decays them with internally controlled rates. The batch size for each training was 8, and since the output was decomposed into nine probability values between 0 and 1, we used a binary cross-entropy loss function to make the gradients decay proportional to errors.

The architecture is shown in Figure 2. Since we could not fully fit the Xception network, we have used convolutional blocks to show the three sections of the network, scaled according to the length of the flows and the input size.

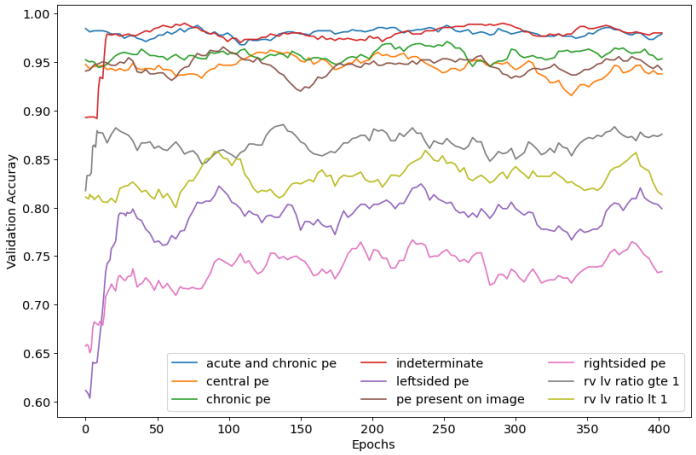

Fig. 3: Validation Accuracy for all metrics

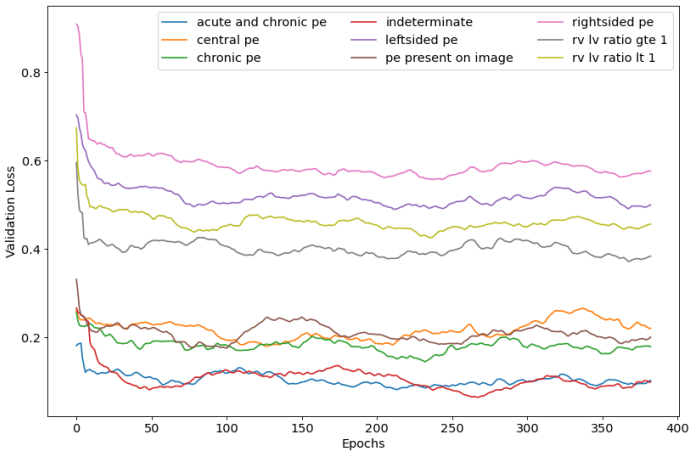

Fig. 4: Validation Losses for all metrics

V. Experiments and Results

Our experimentation pipeline was highly limited to the metrics and time limits of kernels on Kaggle as the data for this competition was around 1 TB, and our systems were not capable of handling or training it. Thus, our model was trained with an upper limit of 4 or 5 hours of training on Kaggle GPUs (40 hours of GPU usage per week), which gave us the ability to train at most 400 epochs in any given run. Our metric of measurement here was the loss over-training and validation and the classification accuracy of our model as measurement metrics. We specifically tracked the accuracy and saved checkpoints to see if the model can improve its accuracy towards the end of the training. In all experiments, the accuracy saturated to a value towards the end of training on the validation set for all labels. Since the problem was decomposed into 9 binary classification problems, we decided to use Binary Cross-Entropy loss while training our model and measured the accuracy between the predicted labels and the validation set labels.

Our baseline was the vanilla architecture described in section IV which presented decent results on the validation set, with a loss of 1.85199 and an average accuracy, equally-weighted for all metrics, at around 89.6%. To beat this baseline, we tried multiple additions. The first modification was increasing the dense layers, but this resulted in decreased performance on the validation set and tended towards over-fitting. Our next approach was to change Xception to Efficentnets [17] as a feature extractor, but this gave us noisy outputs. To still make it work with efficient-nets, we tried to apply multiple augmentations to the dataset with a progressive learning rate, but we were still not able to improve the validation accuracy. Then, We tried Mixup [23] and Cutmix [22] augmentations, but they ended up decreasing our training metrics too. Thus, we decided to stick to our current architecture and went ahead with further experiments using it.

Since the amount of training data was not an issue, we decided to work with a single validation split of 80% per batch of training and tested out hyper-parameter tuning on this pipeline. We first tested the effect of learning rate: We decreased the learning rate to 0.0001 and 0.0005 and these brought down accuracies across all metrics by as much as 5%, while increasing the loss to the ballpark of 2.33 on the validation set. Increasing the learning rate to 0.005 reduced the generalization performance on the validation set, albeit not by more than 1.5%. Thus, we continued with a learning rate of 0.001. Our next attempt was to add random cropping to the prepossessing phase, producing a 320x320 cropped image out of the 512x512 original images. But this led to more noise in the outputs. We also tried cut-mix and mixup on this pipeline, but the results were sub-optimal. What individually improved the performances were rotation and windowing, which we tried with a variety of angles and windows, finally settling on a combined random rotation of 8 degrees and 3-channel window mentioned in III. To improve the performance on the location of PE and the ventricle ratios, we implemented the random horizontal flip augmentation to the image, which was the addition that finally allowed us the beat the baseline.

The loss for the whole duration of the training and during validation has been shown in figure 5. In the end, the overall loss on the validation set was 1.79224, and when compared to the baseline, the loss on all metrics except chronic PE was less than the baseline. This model's unweighted mean accuracy was 92%, which beat the baseline's 89%, while the individual accuracy improved on all metrics except chronic PE, on which this model predicted a validation accuracy of 98% while the baseline had an accuracy of 99%. Figures 4 and 3 show the validation losses and accuracies for each metric over the whole training period. These graphs have been smoothed by a running mean of 20 epochs. One central thing that we noticed through all our runs was that the model always performed relatively worse on the ventricle ratios. Due to the time limit, we could not improve the performance of this metric.

The generalization performance is tested out by Kaggle using

Label	Weight
Negative for PE	0.0736196319
Indeterminate	0.09202453988
Chronic	0.1042944785
Acute & Chronic	0.1042944785
Central PE	0.1877300613
Left PE	0.06257668712
Right PE	0.06257668712
RV/LV Ratio >= 1	0.2346625767
RV/LV Ratio <1	0.0782208589

TABLE I: Final

the private dataset. The standard procedure is to train the model and then use it on the predictions set to generate a submissions file. This submissions file is submitted along with the kernel to be run by Kaggle on the private dataset, which is used to calculate scores. For this competition, Kaggle uses a weighted log loss score on the study and image-level labels. Equation 1 shows the loss calculation for the study level labels, where the loss L_{ij} for study i and label j are computed by multiplying the target of the label y_{ij} by the log of the probability predicted by the model for that label. The loss is weighted by the weight w_{ij} of the label. The weights are specified in [11].

$$L_{ij} = -w_j \times [y_{ij} \times log(p_{ij}) + (1 - y_{ij}) \times log(1 - p_{ij})] \quad (1)$$

Equation 2 shows the loss calculation for the image-level labels, where the weight w is 0.07361963 and q_i is the proportion of the positive to the negative pe images in the dataset used. The exam is denoted by i and k is the identifier for images in the exam.

$$L_{ik} = -[w \times p_i] \times [y_{ik} \times log(p_{ik}) + (1 - y_{ik}) \times log(1 - p_{ik})] \quad (2)$$

One of the rules for submitting to this competition was the notebooks to not use the internet, and since our notebook downloaded the Imagenet weights from the internet, we could not make a full submission. However, we worked around it by first training our model independently and then downloading the network file and re-uploading it as a dataset to be able to make an unofficial and private submission. This allowed us to see how our model performed on the private dataset. There are two scores calculated: The public score is calculated on approximately 28% of data, and the private score is evaluated on 72% of the test dataset. Our model had a private score of 0.899 and a public score of 0.924. Thus, the generalization performance of our model, while not close to the top submissions, was not very far behind in terms of performance. Unfortunately, these tests are solely run on the Kaggle backend, so we could not extract the data during the runs.

VI. Conclusion and Future Work

In this report, we presented an end-end method to detect PE from CT scans of the chest. We worked with the data provided by the Radiological Society of North America, and the model consisted of using the Xception CNN to extract the latent features from the images, on which we applied a classifier to predict the 9 labels of the classification task. These labels were predicted by decomposing the multi-category classification problem into 9 binary classification units, which were used to evaluate the model using accuracy as a measure. The model was able to achieve an accuracy of 92% on the validation test set. While this is not good enough for direct application in the medical field yet, we present an architecture that can be tuned further to increase the classifications' quality. As identified, the avenues in which further improvements can be made are two-fold: First, the feature extractor can be ensembled with RNN units treating the set of images in a study as a series and improve the accuracy, and the data can be further pre-processed to create a better balance between the labels.

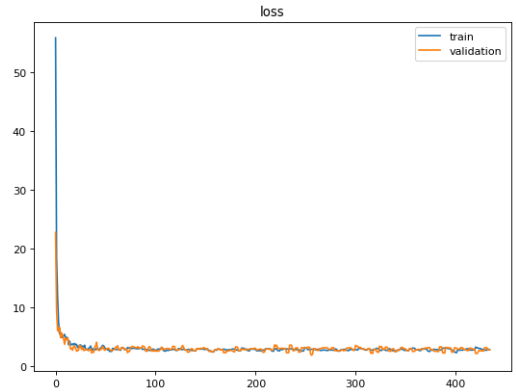

Fig. 5: Overall Loss on Training and Validation Sets

Acknowledgment and Contributions

The completion of any project depends upon cooperation, coordination, and combined efforts of several sources of knowledge. We are immensely thankful to Dr. Maria Zuluaga for the opportunity to work on a project of such nature. We would also like to thank the Radiological Society of North America for the high quality and quantity of data, and the Kaggle platform for creating a simple and intuitive interface to work with this data and generate feedback through scores. We both contributed equally to the work. For sure, at different stages of the project both had the responsibility to research the possible solutions or participate in the development of the kernels, Aditya's focus was more on the concept design and evaluation, while Omid's focus was on creating the overall structure of the code and developing the final Kernels.

References

[1] Mahmoud Al-Ayyoub et al. "Machine learning approach for brain tumor detection". In: Apr. 2012. DOI: 10.1145/2222444.2222467.

[2] Dean Bidgood et al. "Understanding and Using DICOM, The Data Interchange Standard for Biomedical Imaging". In: *Journal of the American Medical Informatics Association : JAMIA* 4 (May 1997), pp. 199–212. DOI: 10.1136/jamia.1997.0040199.

[3] F. Chollet. "Xception: Deep Learning with Depthwise Separable Convolutions". In: *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017, pp. 1800–1807. DOI: 10.1109/CVPR.2017.195.

[4] *CTPA*. https://radiopaedia.org/articles/ct-pulmonary-angiogram-technique.

[5] Manas Das and Lipi Mahanta. "Lung segmentation from CT images: Impact of different window settings on the accuracy of segmentation". In: (Dec. 2018).

[6] J. Deng et al. "ImageNet: A Large-Scale Hierarchical Image Database". In: *CVPR09*. 2009.

[7] *EDA for Data*. https://www.kaggle.com/nitindatta/pulmonary-embolism-dicom-preprocessing-eda.

[8] Geoffrey E. Hinton et al. "Improving neural networks by preventing co-adaptation of feature detectors". In: *ArXiv* abs/1207.0580 (2012).

[9] G. Hounsfield. "Computed Medical Imaging". In: *Journal of Computer Assisted Tomography* 4 (1980), pp. 665–674.

[10] *Insights on Labels*. https://www.kaggle.com/prk007/insights-from-tabular-and-image-data.

[11] *Kaggle Evaluation Criterion*. https://www.kaggle.com/c/rsna-str-pulmonary-embolism-detection/overview/evaluation.

[12] Diederik Kingma and Jimmy Ba. "Adam: A Method for Stochastic Optimization". In: *International Conference on Learning Representations* (Dec. 2014).

[13] Jianming Liang and Jinbo Bi. "Computer Aided Detection of Pulmonary Embolism with Tobogganing and Mutiple Instance Classification in CT Pulmonary Angiography". In: vol. 20. Feb. 2007, pp. 630–41. ISBN: 978-3-540-73272-3. DOI: 10.1007/978-3-540-73273-0_52.

[14] *Radiological Society of North America*. https://www.rsna.org/.

[15] D. Ravì et al. "Deep Learning for Health Informatics". In: *IEEE Journal of Biomedical and Health Informatics* 21.1 (2017), pp. 4–21.

[16] Nima Tajbakhsh, Michael B. Gotway, and Jianming Liang. "Computer-Aided Pulmonary Embolism Detection Using a Novel Vessel-Aligned Multi-planar Image Representation and Convolutional Neural Networks". In: *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*. Ed. by Nassir Navab et al. Cham: Springer International Publishing, 2015, pp. 62–69.

[17] Mingxing Tan and Quoc Le. *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. May 2019.

[18] G. Tourassi et al. "Acute pulmonary embolism: artificial neural network approach for diagnosis." In: *Radiology* 189 2 (1993), pp. 555–8.

[19] Byung Mook Weon et al. "Phase contrast X-ray imaging". In: *International Journal of Nanotechnology* 3 (Jan. 2006). DOI: 10.1504/IJNT.2006.009584.

[20] *windowing explained*. https://www.kaggle.com/redwankarimsony/ct-scans-dicom-files-windowing-explained.

[21] Xin Yang et al. "A Two-stage Convolutional Neural Network for Pulmonary Embolism Detection from CTPA Images". In: *IEEE Access* PP (June 2019), pp. 1–1. DOI: 10.1109/ACCESS.2019.2925210.

[22] Sangdoo Yun et al. "CutMix: Regularization Strategy to Train Strong Classifiers With Localizable Features". In: *2019 IEEE/CVF International Conference on Computer Vision (ICCV)* (2019), pp. 6022–6031.

[23] Hongyi Zhang et al. "mixup: Beyond Empirical Risk Minimization". In: *ArXiv* abs/1710.09412 (2018).